

CEN4025C Module 5 Project

Project Overview

For this project we will implement and execute Cypress End-to-End (E2E) tests for the Task Manager front end.

We will configure Cypress, mock backend HTTP requests, and write an automated E2E test that validates the behavior of one component of the application as a user would experience it.

Note: the Task Manager backend is not required to run since API calls are mocked.

1. Clone and Run the Application

1. Clone the repository.
2. Navigate into the TaskManagerFrontEnd directory.
3. Install dependencies:

npm install

4. Start the Angular application:

npx ng serve

Access the application (<http://localhost:4200>) and ensure it loads successfully before proceeding.

2. Install Cypress

1. Install Cypress and testing utilities as development dependencies:

npm install -D cypress @testing-library/cypress

2. Add the following script lines to package.json (watch your trailing commas, this is a frequent cause of errors):

```
"cypress": "npx cypress open",  
"cypress:run": "npx cypress run"
```

3. Initialize Cypress

1. Verify the application is still running from Step 1, open a new Command Tool, navigate into the TaskManagerFrontEnd directory and run

npm run cypress

2. From the Cypress splash page select:

E2E Testing

and choose a browser (Chrome recommended)

3. Stop Cypress after it initializes (Ctrl + C).

4. Configure TypeScript Support

1. Inside the cypress/ folder, create a new tsconfig.json file (we need this inside the cypress/ folder for the cypress TypeScript configuration settings)
2. Configure it to extend the root Angular TypeScript configuration and include Cypress types:

```
{
  "extends": "../tsconfig.json",
  "compilerOptions": {
    "types": ["cypress", "@testing-library/cypress"],
    "isolatedModules": false,
    "allowJs": true,
    "experimentalDecorators": true,
    "skipLibCheck": true
  },
  "include": ["**/*.ts"],
  "exclude": []
}
```

5. Update Cypress Configuration

1. Edit the cypress.config.ts in the top level project folder and replace its contents with this:

```
import { defineConfig } from "cypress";

export default defineConfig({
  e2e: {
    baseUrl: "http://localhost:4200",
    video: false,
    setupNodeEvents(on, config) {
      // implement node event listeners here
    },
  },
});
```

6. Add Test Selectors

We will make the login component testable by adding data-cy attributes to several elements. Modify the src/app/login/login.component.html as follows:

- Update the <h2> to:

```
<h2 data-cy="login-title">Login</h2>
```

- Modify the username as follows:

```
<input name="username"
[(ngModel)]="username"
required
data-cy="username" />
```

- Modify the password as follows:

```
<input name="password"
type="password"
[(ngModel)]="password"
required
data-cy="password" />
```

- Finally, modify the Submit button as follows:

```
<button type="submit"
data-cy="submit-login">
Login
</button>
```

7. Execute Tests

- Start Angular:

```
npx ng serve
```

- Run the test spec:

```
npx cypress run --spec "cypress/e2e/login.cy.ts"
```

- Look for this output and take a screenshot:

```
Running:  login.cy.ts
(1 of 1)
Login
  ✓ logs in with any non-empty username/password (mock login) (1023ms)
  ✓ shows an error if username/password are empty (189ms)
2 passing (1s)
(Results)
```

Tests:	2
Passing:	2
Failing:	0
Pending:	0
Skipped:	0
Screenshots:	0
Video:	false
Duration:	1 second

Spec Ran: login.cy.ts

(Run Finished)

Spec	Tests	Passing	Failing	Pending	Skipped
------	-------	---------	---------	---------	---------

✓ login.cy.ts 00:01	2	2	-	-	-
---------------------	---	---	---	---	---

✓ All specs passed!	00:01	2	2	-	-
---------------------	-------	---	---	---	---

Submission

Submit your modified project and screenshot of your successful test run to the GitHub Classroom repository.